자료구조 시험
완벽 이해 노트

8장 트리 · 9장 우선순위 큐 · 10장 그래프 I · 11장 그래프 II
강의 PDF 쪽수 표시 · 중간 과정 표 · 내가 틀렸던 포인트 · 최종 모의고사

8장 트리 | 9장 우선순위 큐 | 10장 그래프 I | 11장 그래프 II

모든 중간값과 실행 결과는 강의 코드를 실제로 컴파일·실행해서 확인한 값입니다.
쪽수 표기 `트리 p.39` = 해당 강의 PDF를 뷰어로 열었을 때의 페이지 번호입니다.

목차

1장 8장 트리 — 용어, 이진 트리 성질, 표현, 순회, 레벨 순회, 수식 트리, 스레드, 이진 탐색 트리

2장 9장 우선순위 큐 — 히프 삽입·삭제, 히프 정렬, LPT, 허프만 코드

3장 10장 그래프 I — 용어, 표현, DFS, BFS, 연결 성분, "경로가 존재하는가" 문제

4장 11장 그래프 II — 신장 트리, Kruskal, Union-Find, Prim, Dijkstra, Floyd, 위상 정렬

5장 총정리 — 헷갈리는 짝, 복잡도표, 빈칸 단골 코드와 쪽수, 오답 노트, 최종 모의고사와 정답

이 노트 사용법과 시험 전략

이 노트를 보는 순서

1. **개념 설명**을 읽으면서 옆에 강의 PDF의 `쪽수`를 펴 놓고 그림을 같이 봅니다.
2. **중간 과정 표**는 눈으로만 보지 말고 **빈 종이에 직접 표를 그려** 따라 채운 뒤 비교합니다. 시험이 바로 이 표를 묻습니다.
3. **빨간 상자(함정)**와 **노란 상자(내가 틀렸던 것)**는 시험 전날 다시 봅니다.
4. 마지막 장의 **최종 모의고사**를 시간을 재고 풀고, 정답은 맨 뒤에서 확인합니다.

시험 특징 (선배들 후기 정리)

특징	대비 방법
객관식 · 주관식 · T/F · 코드 빈칸	T/F 단골 문장은 각 장의 **T/F 모음**에 정리
결과값보다 **중간 과정**을 물음	단계마다 **배열 전체를 다시 적는 표**로 연습 (이 노트의 모든 표)
변수명만 바꾼 코드, 교재와 **다른 방식**의 코드	"다른 방식" 상자: union 방향, BST 삭제 방식, 허프만 0/1, 스택↔큐
빈칸 문제는 교재 코드 그대로 (변수명 안 바뀜)	5장 **빈칸 단골 코드 + 쪽수 표**로 빨리 찾기
시간이 부족함 (오픈북)	아래 시간 배분 순서대로 풀기

시간 배분 전략

순서	문제 유형	이유
1	T/F, 개념 객관식	문장만 읽고 바로 판단. 1문제 1분 이내
2	코드 빈칸	교재 쪽수만 찾으면 답이 있음 (5장 표 이용)
3	결과값 (순회 결과, MST 합)	빠른 방법(테두리 따라 점 찍기 등)으로 계산
4	중간 과정 (배열·큐·스택 변화)	표를 그려야 해서 시간이 걸림 → 남은 시간에
5	복잡한 개념 서술	부분 점수를 노리고 핵심 문장부터 쓰기

시험장 5대 체크리스트 (내가 실제로 틀렸던 유형)

1. **"직후"가 어느 시점인지** 표를 그려 정확히 맞춘다. 한 단계 더 가거나 덜 가지 않는다.
2. **union은 대장(루트)끼리.** 바뀌는 칸은 `parent[첫 번째 정점의 대장]`, 적는 값은 **두 번째 정점의 대장**.
3. **짝 구분:** Prim(`weight`) ↔ Dijkstra(`distance[u] + weight`), Floyd `A`(거리) ↔ `next`(정점 번호), 중간값 ↔ 최종값.
4. **빈칸은 문법까지:** `g->`는 구조체 안 멤버(`g->weight`, `g->n`)에만, 전역 배열(`distance`, `visited`, `parent`)에는 안 붙음. 변수 이름(v인지 w인지)은 주변 코드 그대로.
5. **답을 두 개 묻는 문제는 둘 다** 쓴다 (교재 방식 / 다른 방식, 값 / 이유).

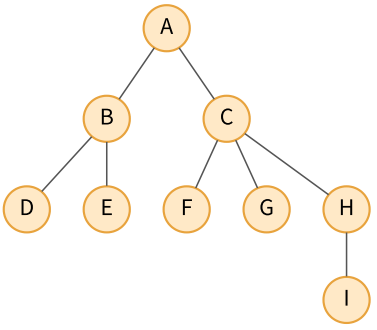
1장 · 8장 트리

강의 PDF「8장 트리」(92쪽) 기준

1. 트리 용어 트리 p.6~9

트리는 **계층 구조**입니다. 노드들이 **부모–자식 관계**로 연결되어 있어요. 리스트·스택·큐는 일렬(선형) 구조라는 점과 대비됩니다.

용어	뜻	p.9 트리에서
루트	부모가 없는 노드	A
단말 노드	**자식이 없는** 노드	D, E, F, G, I (**5개**)
비단말 노드	자식이 하나라도 있는 노드	A, B, C, H
노드의 차수	그 노드의 **자식 수**	B=2, C=3, H=1
트리의 차수	노드 차수 중 **최댓값**	3
레벨	층 번호, **루트가 1**	I는 레벨 4
높이	트리의 최대 레벨	4



트리 p.9 예제 트리

> **내가 틀렸던 것: 노드 수 ≠ 단말 노드 수**
> "단말 노드 개수"를 묻는데 전체 노드 수(9개)를 적었음. 단말 노드 = **자식이 없는 노드만** → 5개. 문제의 단어를 꼭 확인!

> **함정**
> - 이 교재는 **레벨을 1부터** 셉니다 (다른 책은 0부터인 경우도 있음).
> - **트리의 차수**는 노드 차수의 **합이 아니라 최댓값**.
> - 차수는 **자식 수만** 셉니다 (부모로 가는 간선은 안 셈).

2. 이진 트리의 성질 트리 p.11~18

이진 트리 = 모든 노드의 자식이 **최대 2개**이고, **왼쪽/오른쪽을 구분**하는 트리. **공집합도 이진 트리**입니다(재귀적 정의).

성질	공식	왜 그런가	예
노드 n개 → 간선 수	$n - 1$	루트 빼고 모든 노드는 부모로 가는 간선이 정확히 1개	n=7 → 6
높이 h → 노드 수	$h \le \text{노드} \le 2^h - 1$	레벨 i에 최대 2^{i-1}개, 합하면 2^h-1	h=5 → 5~31
노드 n개 → 높이	$\lceil \log_2(n+1) \rceil \le h \le n$	최소는 꽉 채울 때, 최대는 경사 트리	n=20 → 최소 $\lceil \log_2 21 \rceil = 5$

	포화 이진 트리	완전 이진 트리
정의	모든 레벨이 **꽉 참**	마지막 레벨 직전까지 꽉 차고, 마지막 레벨은 **왼쪽부터 빈칸 없이**
노드 수	정확히 2^h-1	2^{h-1} ~ 2^h-1

T/F 단골

포화 → 완전: **T**

완전 → 포화: **F**

완전 이진 트리는 포화 트리와 노드 번호가 일치: **T**

마지막 레벨은 오른쪽부터 채워도 됨: **F**

3. 이진 트리 표현 트리 p.20~27

배열 표현 (인덱스 1부터)

구할 것	공식	i=6일 때
부모	**i / 2** (정수 나눗셈)	3
왼쪽 자식	**2i**	12
오른쪽 자식	**2i + 1**	13

K진 트리(p.22~23, 보충): 부모 (i+(K−2))/K, 맨 왼쪽 자식 Ki−(K−2), 맨 오른쪽 자식 Ki+1. K=2를 넣으면 위 공식.

경사 트리는 낭비가 큼: p.20 (b)의 A,B,C,D는 인덱스 1,2,4,8 → 노드 4개에 8칸.

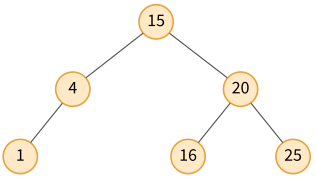
링크 표현 트리 p.25

```
typedef struct TreeNode {
    int data;
    struct TreeNode *left, *right;
} TreeNode;
```

자식이 없는 쪽은 `NULL`.

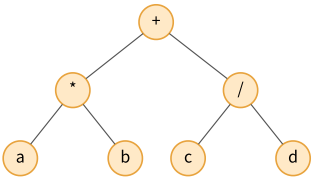
4. 순회: 전위·중위·후위 트리 p.28~43

순회	순서	루트 방문 시점	응용
전위	**V**LR	맨 처음	구조화된 문서 출력 p.32
중위	L**V**R	가운데	수식 트리 p.35
후위	LR**V**	맨 끝	**디렉토리 용량** p.38



트리 p.39

p.39 트리: 전위 `15 4 1 20 16 25` / 중위 `1 4 15 16 20 25` (이진 탐색 트리라 오름차순) / 후위 `1 4 16 25 20 15`



수식 트리 (a*b)+(c/d)

수식 트리 (a*b)+(c/d): 전위 `+ * a b / c d` / 중위 `a * b + c / d` / 후위 `a b * c d / +`

빠르게 구하는 법: 테두리 따라 점 찍기

트리 바깥을 루트 왼쪽에서 출발해 한 바퀴 돈다고 생각하고, 각 노드를 **왼쪽에서 처음 지날 때** 적으면 전위, **아래를 지날 때** 적으면 중위, **오른쪽에서 지날 때** 적으면 후위.

내가 틀렸던 것: 중위 순회에서 루트 위치

`a*bc/d+` 로 써서 루트 +가 맨 끝으로 감(후위와 섞임). 중위는 **루트가 항상 왼쪽 서브트리와 오른쪽 서브트리 사이**. 수식 트리를 중위 순회하면 **원래 수식이 그대로** 나와야 함 → 검산.

반복적 중위 순회: 스택 변화 트리 p.44~45

```
while (1) {
    for (; root; root = root->left)  // ① 왼쪽 끝까지 push
        push(root);
    root = pop();                    // ② 꺼내서
    if (!root) break;                //    비었으면 끝
    printf("[%d] ", root->data);     // ③ 출력
    root = root->right;              // ④ 오른쪽으로
}
```

동작 (p.39 트리)	스택(오른쪽=top)	출력
15, 4, 1 push	[15, 4, 1]	
pop 1, 오른쪽 NULL	[15, 4]	1
pop 4, 오른쪽 NULL	[15]	4
pop 15, 오른쪽 = 20	[ ]	15
20, 16 push	[20, 16]	
pop 16	[20]	16
pop 20, 오른쪽 = 25	[ ]	20
25 push → pop	[ ]	25

수식 트리 (a*b)+(c/d)에서 처음 a가 출력된 직후 스택 = [+, *]. 반복 전위 순회(p.46)는 **오른쪽 자식을 먼저 push** (나중에 넣은 왼쪽이 먼저 나오도록).

5. 레벨 순회 (큐) 트리 p.50~56

```
enqueue(&q, ptr);
while (!is_empty(&q)) {
    ptr = dequeue(&q);
    printf(" [%d] ", ptr->data);
    if (ptr->left)  enqueue(&q, ptr->left);
    if (ptr->right) enqueue(&q, ptr->right);
}
```

꺼낸 노드 (p.39 트리)	넣은 노드	큐 (왼쪽=front)
(시작)	15	[15]
15	4, 20	[4, 20]
4	1	**[20, 1]**
20	16, 25	[1, 16, 25]
1, 16, 25	없음	[ ]

결과 `15 4 20 1 16 25`. 전위·중위·후위는 **스택(재귀)**, 레벨 순회는 **큐**.

내가 틀렸던 것: "4를 처리한 직후" 큐

[20, 1, 16, 25]라고 씀 → 한 단계(20 처리)를 더 진행한 상태. 정답 **[20, 1]**. "직후"는 **그 노드의 자식을 다 넣은 순간**까지만.

6. 수식 트리 · 디렉토리 용량 · 트리 연산 트리 p.57~66

수식 트리 계산 = 후위 순회. 양쪽 자식 값을 먼저 알아야 연산 가능. p.60~62 실행 결과, 트리 (1*4)+(16+25):

```
1 * 4을 계산합니다.      ← 왼쪽 서브트리
16 + 25을 계산합니다.    ← 오른쪽 서브트리
4 + 41을 계산합니다.     ← 루트
수식의 값은 45입니다.
```

디렉토리 용량 = 후위 순회. 노드 값 = 자기 크기 + 왼쪽 + 오른쪽. n3 = 100+500+200 = 800, n1 = 0+50+800 = **850** p.64

함수	NULL	단말	그 외
get_node_count	0		**1** + 왼 + 오
get_leaf_count	0	**1**	왼 + 오 (1 안 더함)
get_height	0		**1 + max**(왼, 오)

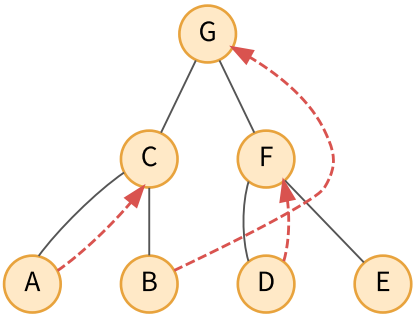
p.39 트리: 노드 6, 단말 3, 높이 3.

7. 스레드 이진 트리 트리 p.67~70

문제의식: 노드 n개 이진 트리의 링크는 2n개, 실제 간선은 n−1개 → **n+1개 링크가 NULL로 낭비**.

아이디어: 오른쪽 칸이 빈 노드에 "**중위 순회에서 내 다음 노드(중위 후속자)**"를 적어 둔다. 이 메모가 **스레드**(그림의 빨간 점선).

중위 순서: **A C B G D F E** → 스레드: A→C, B→G, D→F. E는 마지막이라 NULL.



트리 p.67 (점선 = 스레드)

노드	right가 가리키는 곳	정체	is_thread
A	C	스레드 (메모, 위로 올라가는 점선)	**1**
B	G	스레드	**1**
C	B	진짜 자식 (실선)	0
D	F	스레드	**1**
E	NULL	비어 있음	0
F	E	진짜 자식	0
G	**F**	**진짜 자식** (D의 right도 F지만 의미가 다름!)	0

```
TreeNode *find_successor(TreeNode *p) {
    TreeNode *q = p->right;
    if (q == NULL || p->is_thread == TRUE)
        return q;              // 규칙 1: 메모면 그대로
    while (q->left != NULL)
        q = q->left;           // 규칙 2: 진짜 자식이면
    return q;                  //   오른쪽 서브트리의 맨
왼쪽
}
```

현재	규칙	다음
A	1 (스레드)	C
C	2: B, 왼쪽 없음	B
B	1	G
G	2: F → 왼쪽 끝 D	**D**
D	1	F
F	2: E, 왼쪽 없음	E
E	NULL	끝

왜 진짜 자식이면 맨 왼쪽까지 내려가나?

중위 순회는 L→V→R. G를 방문(V)했으면 다음은 G의 오른쪽 서브트리(R)인데, 그 안에서도 L이 먼저라 **가장 왼쪽 노드(D)**가 첫 방문. `while`이라 **여러 단계** 내려갈 수 있음(`if`가 아님).

내가 틀렸던 것: find_successor(G) = F?

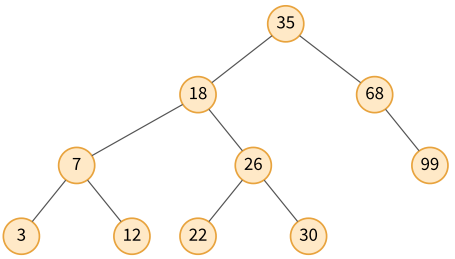
G의 right(F)는 스레드가 아니라 **진짜 자식** → 규칙 2 → 정답 **D**. 그림에서 **위로 올라가는 점선 = 스레드, 아래로 내려가는 실선 = 진짜 자식**. 스레드 트리는 **스택·재귀 없이** 중위 순회 가능(T/F 단골). `is_thread`는 **오른쪽** 링크 표시.

8. 이진 탐색 트리 (BST) 트리 p.71~92

정의: 왼쪽 서브트리 키 ≤ 루트 키 ≤ 오른쪽 서브트리 키. **중위 순회 = 오름차순.**

탐색 p.72~76: 같으면 성공, 작으면 왼쪽, 크면 오른쪽. p.80 트리에서 12 탐색: 35 → 18 → 7 → 12 (비교 4번).

삽입 p.77~79: 탐색하다 **실패한 자리(NULL)**에 붙임. 같은 키는 삽입 안 함. 정렬된 순서로 넣으면 **경사 트리**.



트리 p.80 (35,18,68,7,26,99,3,12,22,30 순 삽입)

```
TreeNode *insert_node(TreeNode *node, int key) {
    if (node == NULL) return new_node(key);
    if (key < node->key)      node->left  = insert_node(node->left, key);
    else if (key > node->key) node->right = insert_node(node->right, key);
    return node;
}
```

삭제: 세 가지 경우 p.80~86

경우	예 (p.80 트리)	처리
1. 단말 노드	30 삭제	부모 링크를 NULL로
2. 자식 하나	68 삭제	자식(99)을 부모(35)에 바로 연결
3. 자식 둘	18 삭제	**오른쪽 서브트리 최솟값(22)**으로 교체 후 22 삭제

```
if (root->left == NULL)  { TreeNode *temp = root->right; free(root); return temp; }
else if (root->right == NULL) { TreeNode *temp = root->left; free(root); return temp; }
TreeNode *temp = min_value_node(root->right);        // 세 번째 경우
root->key = temp->key;                                 // 후계 노드 복사
root->right = delete_node(root->right, temp->key);     // 후계 노드 삭제
```

> **교재와 다른 방식**
> 자식이 둘일 때 **왼쪽 서브트리의 최댓값**을 쓰는 방식도 있음(p.83). 18 삭제 시 교재 방식은 **22**, 다른 방식은 **12**. 문제가 어느 방식인지 확인하고, 묻는 경우 **둘 다** 쓰기. 빈칸: "조건에서 NULL인 쪽의 **반대쪽**을 반환".

성능 p.89~91: 탐색·삽입·삭제 모두 **O(h)**. 균형이면 $h = \lceil \log_2(n+1) \rceil \rightarrow O(\log n)$, 경사면 $h = n \rightarrow$ **O(n)**. **영어 사전**(p.92)은 키가 문자열이라 `strcmp`로 비교.

p.88 main: 30,20,10,40,50,60 순 삽입 → 높이 4 (30→40→50→60).

2장 · 9장 우선순위 큐

강의 PDF「9장 우선순위 큐」(54쪽) 기준

1. 우선순위 큐와 구현 방법 우선순위큐 p.2~7

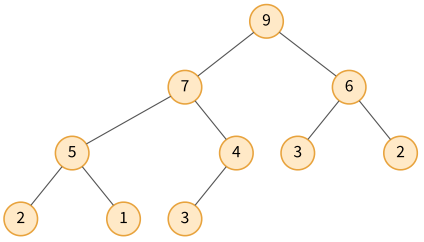
일반 큐는 FIFO, 우선순위 큐는 **우선순위가 높은 것이 먼저** 나감(응급실). 핵심 연산은 **insert**와 **delete**.

구현 방법	삽입	삭제
정렬 안 된 배열 / 연결 리스트	O(1)	O(n)
정렬된 배열 / 연결 리스트	O(n)	O(1)
히프	**O(log n)**	**O(log n)**

2. 히프 우선순위큐 p.8~13

- **완전 이진 트리**이면서 **최대 히프**: 부모 ≥ 자식 / **최소 히프**: 부모 ≤ 자식.
- **부모–자식 사이만** 크기 관계. **형제 사이는 순서 없음**(느슨한 정렬).
- **같은 키 값이 여러 개 있어도 됨**(BST와 다른 점).
- 높이 = $\lceil \log_2(n+1) \rceil$ p.10
- 배열로 구현: 인덱스 1부터, 부모 i/2, 왼쪽 2i, 오른쪽 2i+1.

```
인덱스: 1  2  3  4  5  6  7  8  9  10
값:     9  7  6  5  4  3  2  2  1  3
```



우선순위큐 p.11의 최대 히프

3. 삽입: upheap 우선순위큐 p.14~18

```
i = ++(h->heap_size);                    // 맨 끝 자리
while ((i != 1) &&
       (item.key > h->heap[i / 2].key)) {
    h->heap[i] = h->heap[i / 2];         // 부모를 아래로
    i /= 2;                              // 한 칸 위로
}
h->heap[i] = item;                       // 마지막에 한 번만 저장
```

단계 (8 삽입)	[2]	[5]	[11]
시작: 11번 자리	7	4	(8)
8 > 4 → 4를 내림	7	(8)	**4**
8 > 7 → 7을 내림	(8)	**7**	4
8 < 9 → 멈춤, 저장	**8**	**7**	4

결과 [9, **8**, 6, 5, **7**, 3, 2, 2, 1, 3, **4**]. 교환(swap)이 아니라 **부모를 내리기만** 하고 새 값은 **마지막에 한 번** 저장.

> **내가 틀렸던 것: 지나간 길의 값이 밀려 내려감**
> "인덱스 5의 값"을 원래 값 4로 적음 → 8이 지나가면서 7이 5번으로 내려옴. 정답 **7**. 단계마다 **배열 전체를 다시 적기**.

4. 삭제: downheap 우선순위큐 p.19~23

```
item = h->heap[1];                      // 꺼낼 루트
temp = h->heap[(h->heap_size)--];       // 마지막 노드
parent = 1; child = 2;
while (child <= h->heap_size) {
    if ((child < h->heap_size) &&
        (h->heap[child].key) < h->heap[child+1].key)
        child++;                        // 더 큰 자식
    if (temp.key >= h->heap[child].key) break;
    h->heap[parent] = h->heap[child];  // 자식을 위로
    parent = child;  child *= 2;
}
h->heap[parent] = temp;
```

3의 위치	두 자식	큰 자식	결과
1	7, 6	7	내려감
2	5, 4	5	내려감
4	2, 1	2	멈춤

원래 히프에서 9 삭제 → 마지막 값 3을 루트로 → 결과 [**7, 5**, 6, **3**, 4, 3, 2, 2, 1]. `child < heap_size`는 **오른쪽 자식이 있는지** 확인.

내가 틀렸던 것: heap[1]을 1이라고 씀

검산: 최대 히프는 삭제 후에도 **루트 = 남은 값 중 최댓값**이어야 함 → heap[1]은 반드시 7.

최대 히프 ↔ 최소 히프: 부등호만 반대

위치	최대 히프	최소 히프 (LPT, 허프만)
upheap 조건	`item.key > heap[i/2].key`	`item.key < heap[i/2].key`
자식 선택	`heap[child] < heap[child+1]` (큰 쪽)	`heap[child] > heap[child+1]` (작은 쪽)
멈춤 조건	`temp.key >= heap[child].key`	`temp.key < heap[child].key`

5. 히프 정렬 우선순위큐 p.30~33

n개를 모두 최대 히프에 삽입 → 하나씩 삭제하며 **배열 뒤(a[n−1])부터** 채움 → **오름차순**. 복잡도 **O(n log n)**. 가장 큰 값 몇 개만 필요할 때 특히 유리. p.33 결과: `9 11 23 27 34 56 56 99`.

```
for (i = 0; i < n; i++) insert_max_heap(h, a[i]);
for (i = (n - 1); i >= 0; i--) a[i] = delete_max_heap(h);
```

6. LPT 머신 스케줄링 우선순위큐 p.34~39

작업을 **긴 것부터**(미리 정렬) 하나씩, **가장 빨리 비는 기계**에 배정. 그 기계를 찾으려고 **avail(사용 가능 시간)을 키로 하는 최소 히프** 사용.

```
for (i = 0; i < JOBS; i++) {
    m = delete_min_heap(h);
    printf("JOB %d을 시간=%d부터 시간=%d까지 기계 %d번에 할당한다.\n",
           i, m.avail, m.avail + jobs[i] - 1, m.id);
    m.avail += jobs[i];
    insert_min_heap(h, m);
}
```

p.39 예제 (작업 8,7,6,5,3,2,1 / 기계 3대). 표기 (기계, avail), 히프는 배정 후 상태.

작업	길이	꺼낸 기계	시간	배정 후 히프 배열
(시작)				[(1,0), (2,0), (3,0)]
JOB 0	8	1 (0)	0~7	[(2,0), (3,0), (1,8)]
JOB 1	7	2 (0)	0~6	[(3,0), (1,8), (2,7)]
JOB 2	6	3 (0)	0~5	[(3,6), (1,8), (2,7)]
JOB 3	5	3 (6)	6~10	[(2,7), (1,8), (3,11)]
JOB 4	3	2 (7)	7~9	[(1,8), (3,11), (2,10)]
JOB 5	2	1 (8)	8~9	[(2,10), (3,11), (1,10)]
JOB 6	1	**2 (10)**	10~10	

동점 처리

JOB 6에서 기계 1, 2 모두 avail 10. 기계 2는 이미 루트, 기계 1은 나중에 끝에 삽입됐는데 upheap 조건이 `<`라 **같으면 올라가지 않음** → 루트의 기계 2가 뽑힘. 동점은 **히프 배열 상태**로 결정되므로 반드시 배열을 따라가야 함.

내가 틀렸던 것: JOB 번호는 0부터

기계 2대, 작업 6,5,4,3,2 문제에서 JOB 1의 결과를 JOB 2로 적음. 정답: JOB 0 기계1 0~5, JOB 1 기계2 0~4, **JOB 2 기계2 5~8**, JOB 3 기계1 6~8, **JOB 4 기계2 9~10**(동점 9에서 루트의 기계2). 끝나는 시간 = **avail + 길이 − 1**.

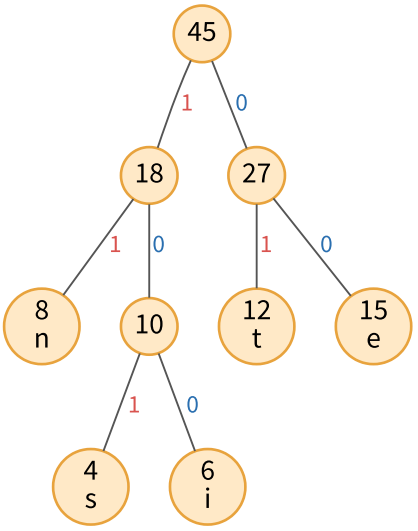
LPT는 **최소 히프**(가장 빨리 비는 기계). "긴 작업부터"는 작업 배열을 **미리 정렬**하는 부분. 둘을 헷갈리지 말 것.

7. 허프만 코드 우선순위큐 p.40~54

자주 나오는 글자일수록 짧은 코드. 만드는 법: 빈도를 키로 **최소 히프**에 넣고, **가장 작은 두 개를 꺼내 합치고 다시 넣기**를 n−1번.

합치기	꺼낸 두 값	결과	남은 히프
1	4(s), 6(i)	10	8, 10, 12, 15
2	8(n), 10	18	12, 15, 18
3	12(t), 15(e)	27	18, 27
4	18, 27	45	45

글자	n	s	i	t	e
교재 코드	11	101	100	01	00
0/1 반대	00	010	011	10	11



교재 print_codes: **왼쪽 = 1**, **오른쪽 = 0**

전체 비트 수 = Σ(빈도 × 코드 길이) = 8×2 + 4×3 + 6×3 + 12×2 + 15×2 = **100**. `make_tree(e1, e2)` 에서 먼저 꺼낸(작은) 값이 **왼쪽**.

p.53~54: 빈도 2,3,5,7,9,13 → 2+3=5, **5+5=10**(합친 5와 원래 5), 7+9=16, 10+13=23, 16+23=39. 비용 **93** < 높이를 낮춘 트리 95 → 허프만이 비용 최소(높이가 낮다고 최소인 게 아님).

연습: A=1, B=2, C=3, D=4 → 1+2=3, 3+3=6, 4+6=10. 코드 길이 D1, C2, A3, B3 → 비트 수 4+6+3+6 = **19**.

> **내가 틀렸던 것: 답을 하나만 씀**
> "교재 코드의 i와, 0/1을 바꾼 i"를 물었는데 011만 씀. **두 개를 묻으면 두 개 다**: 100 / 011.

3장 · 10장 그래프 I

강의 PDF「10장 그래프 I」(45쪽) 기준

1. 그래프 용어 그래프I p.5~16

- **G = (V, E)**. 무방향 간선 (A, B) = (B, A), 방향 간선 <A, B> ≠ <B, A> (괄호 모양으로 구분). 가중치 그래프 = **네트워크**.
- **부분 그래프**: V(S) ⊆ V(G), E(S) ⊆ E(G). **트리** = 사이클 없는 연결 그래프.

개념	공식 / 성질	예
무방향 그래프 차수	**차수 합 = 간선 수 × 2** (간선 하나가 양 끝 차수를 1씩 올림)	p.11: 차수 합 10, 간선 5
방향 그래프	**진입 차수 합 = 진출 차수 합 = 간선 수**	p.12: 둘 다 3
무방향 완전 그래프	**n(n−1)/2** (방향이면 n(n−1))	n=6 → 15
경로의 길이	경로에 쓰인 **간선 수**	

조건	경로	단순 경로
이웃한 정점 사이에 간선이 있어야 함	필요	필요
정점이 반복되면 안 됨	**필요 없음**	필요

p.14 G1 (간선 0-1, 0-2, 0-3, 1-2, 2-3): 0,1,2,3 경로 ⭕ / 0,1,3,2 경로 ❌(1-3 없음) / 1,0,2,3 단순 경로 ⭕ / **1,0,2,0,3 경로 ⭕, 단순 경로 ❌** / 0,1,2,0 사이클 ⭕

> **내가 틀렸던 것: "1,0,2,0,3은 경로인가?" → X라고 씀**
> 경로의 조건은 "이웃 사이에 간선이 있다"뿐 → **경로 O**. 반복 금지는 **단순 경로**에만.

T/F: 차수 합은 항상 짝수 → "차수가 1,2,2인 무방향 그래프" 같은 건 존재 불가(F).

2. 그래프 표현 그래프I p.18~23

	인접 행렬 (p.19)	인접 리스트 (p.20)
메모리	**n^2**	**n + 2e** (무방향)
간선 (i,j) 확인	**O(1)**	리스트 탐색
정점의 차수	행의 합	리스트 길이
유리한 경우	밀집 그래프	희소 그래프
DFS / BFS 복잡도	**$O(n^2)$**	**O(n + e)**

```
/* 인접 행렬 (p.22): 무방향이라 대칭 */
g->adj_mat[start][end] = 1;
g->adj_mat[end][start] = 1;

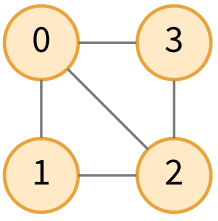
/* 인접 리스트: 맨 앞에 삽입! */
node->vertex = v;
node->link = g->adj_list[u];
g->adj_list[u] = node;
```

리스트 앞 삽입 함정

(0,1), (0,2), (0,3) 순서로 넣으면 정점 0의 리스트는 **3 → 2 → 1** (거꾸로). 같은 그래프의 DFS: 인접 행렬 `0 1 2 3`, 인접 리스트 `0 3 2 1`. 인접 리스트 버전은 **리스트 순서대로** 방문.

3. 깊이 우선 탐색 DFS 그래프I p.26~35

한 방향으로 갈 수 있을 때까지 가다가, 막히면 가장 가까운 갈림길로 **되돌아감(backtracking)**. 되돌아갈 곳을 기억하는 **스택** 필요 → 재귀 호출이 그 역할.



그래프I p.31 main의 그래프

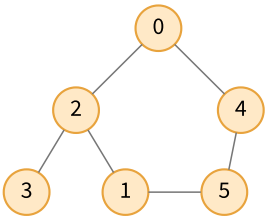
```
void dfs_mat(GraphType* g, int v) {
    int w;
    visited[v] = TRUE;
    printf("정점 %d -> ", v);
    for (w = 0; w < g->n; w++)
        if (g->adj_mat[v][w] && !visited[w])
            dfs_mat(g, w);
}
```

```
dfs(0): 0 방문 → 1로
  └ dfs(1): 1 방문, 0은 방문함 → 2로
     └ dfs(2): 2 방문, 0·1 방문함 → 3으로
        └ dfs(3): 3 방문, 갈 곳 없음 → 복귀        출력 0 1 2 3 (재귀 깊이 최대 4)
```

명시적 스택 버전 (p.34): pop한 정점이 미방문일 때만 방문 표시, 미방문 이웃을 **모두 push**. 재귀처럼 이웃 하나로 바로 들어가지 않음.

4. 너비 우선 탐색 BFS 그래프I p.36~44

시작 정점에서 **가까운 정점부터** 방문. **큐** 사용.



그래프I p.41 간선 목록으로 그린 그래프 (슬라이드에 그림 없음)

```
visited[v] = TRUE; enqueue(&q, v);
while (!is_empty(&q)) {
    v = dequeue(&q);
    for (w = 0; w < g->n; w++)
        if (g->adj_mat[v][w] && !visited[w]) {
            visited[w] = TRUE;   // 넣을 때 바로 표시
            printf("%d 방문 -> ", w);
            enqueue(&q, w);
        }
}
```

꺼낸 정점	넣은 정점	큐 (왼쪽=front)
(시작)	0	[0]
0	2, 4	[2, 4]

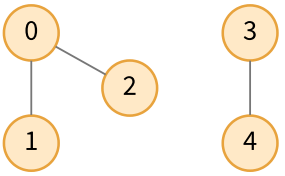
2	1, 3	[4, 1, 3]
4	5	[1, 3, 5]
1	(5는 방문함)	[3, 5]
3, 5		[5] → [ ]

출력 `0 2 4 1 3 5` (p.42). 방문 표시는 **큐에 넣는 순간** → 같은 정점이 두 번 들어가지 않음. p.31 그래프에서 0부터 BFS: 0 처리 직후 큐 [1, 2, 3], 순서 0 1 2 3.

5. 연결 성분 찾기 그래프I p.45

연결 성분 = 서로 경로로 이어진 정점 덩어리(최대로 연결된 부분 그래프). **간선 없는 정점도 혼자서 1개**.

아이디어(색칠 놀이): 아직 안 칠한 정점을 찾을 때마다 새 번호(count++)로 DFS → 그 덩어리 전체가 같은 번호. 끝나면 count = 연결 성분 수.



그래프I p.45

```
void find_connected_component(GraphType *g) {
    count = 0;
    for (i = 0; i < g->n; i++)
        if (!visited[i]) {     // 새 덩어리
            count++;           // 반드시 먼저!
            dfs_mat(g, i);
        }
}
/* dfs_mat 안: */  visited[v] = count;   // TRUE 대신 덩어리 번호 (바뀐 건 이 한 줄)
```

i	상황	count	칠해지는 정점	visited[0..4]
0	미방문 → 새 덩어리	1	0, 1, 2	[1, 1, 1, 0, 0]
1, 2	이미 칠해짐	1		
3	미방문 → 새 덩어리	2	3, 4	[1, 1, 1, 2, 2]

visited 배열의 두 역할

0 = 미방문, 1 이상 = 방문함 + 몇 번째 덩어리. 덩어리 번호가 **1부터** 시작해야 `!visited[w]` 체크가 그대로 동작.

- `find_connected_component`에서 dfs_mat 직접 호출 횟수 = **연결 성분 수**
- 재귀 포함 dfs_mat 전체 호출 횟수 = **정점 수** (정점마다 딱 한 번)
- DFS 대신 BFS를 써도 됨

count++를 dfs_mat 뒤로 옮기면?

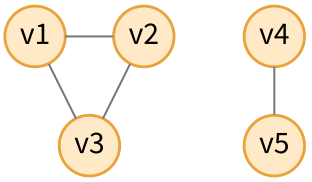
첫 덩어리가 0으로 기록 → 0은 "미방문"이라 방문 체크가 깨짐 → 0과 이웃이 서로를 끝없이 호출 → **무한 재귀**로 비정상 종료. (서술형은 원인과 결과까지)

내가 틀렸던 것

① 빈칸 `visited[v] = ____`에 TRUE → 연결 성분 버전은 **count**. 함수 주변(`count++`)으로 어느 버전인지 먼저 확인. ② 배열을 묻는데 값 하나만 씀 → 정점 7개(간선 0-3, 1-2, 2-5, 3-6) 정답 visited = **[1, 2, 2, 1, 3, 2, 1]**. 번호는 바깥 for문이 **처음 만나는 순서**. ③ 전체 호출 횟수를 9라고 씀 → **정점 수 7**.

6. "v1 → v7 최단 경로가 존재하는가? (T/F)" 해석법

사실은 **"간선을 따라 v1에서 v7에 도착할 수 있는가?"**를 묻는 문제. 길이 하나라도 있으면 최단 경로도 있음 → T.



그래프I p.15 (b) 비연결 그래프

F가 되는 함정	예
① 덩어리가 다름 (연결 성분)	p.15 (b): v1→v5는 F
② 방향 그래프의 화살표 방향	v1→v2→v3←v7: v1→v7은 F, v7→v3은 T
③ 행렬의 999 / ∞	간선 없음으로 처리

푸는 순서: ① 화살표 확인 → ② v1에서 갈 수 있는 정점을 손가락으로 따라가기(머릿속 DFS/BFS) → ③ v7에 닿으면 T. 서술형 근거: Dijkstra 후 distance[v7] = INF, Floyd A[v1][v7] = INF, BFS에서 v7 미방문이면 경로 없음.

4장 · 11장 그래프 II

강의 PDF「11장 그래프 II」(72쪽) 기준 · 시험 비중이 가장 큰 장

1. 신장 트리와 MST 그래프II p.2~5

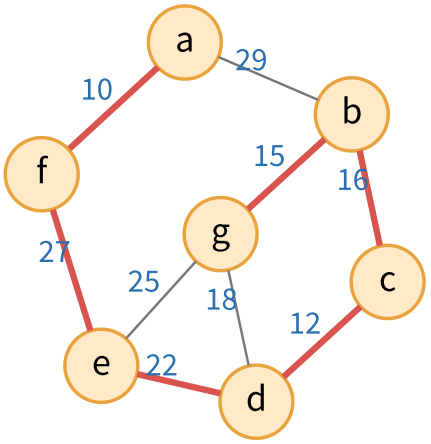
- **신장 트리**: 모든 정점을 포함하는 트리. 간선 **정확히 n−1개**, 사이클 없음. 한 그래프에 **여러 개** 가능. DFS/BFS가 지나간 간선으로 만들 수 있음(p.3).
- **최소 비용 신장 트리(MST)**: 가중치 합이 최소인 신장 트리. p.5: 일반 신장 트리 11, MST 7.
- **MST 가중치 합은 어떤 알고리즘이든 같음**(Kruskal = Prim = 102). 단, **간선을 고르는 순서는 다름**.

2. Kruskal 그래프II p.6~21

탐욕적 방법: 매 순간 최선을 선택. 일반적으로는 최적 보장이 없지만 **Kruskal은 최적임이 증명됨**(p.6, T/F 단골).

1. 간선을 **가중치 오름차순** 정렬
2. 작은 것부터, **사이클이 안 생기면 채택**, 생기면 버림
3. **n−1개 채택하면 종료**

p.8 알파벳 그래프 = p.20 숫자 그래프 (a=0, b=1, c=2, d=3, e=4, f=5, g=6). 빨간 굵은 선 = MST.



그래프II p.8 (빨간 선 = MST)

순서	간선	가중치	결과	이유
1	af	10	채택	
2	cd	12	채택	
3	bg	15	채택	
4	bc	16	채택	{b, c, d, g}가 한 덩어리
5	dg	18	버림	d, g가 이미 같은 덩어리 → 사이클
6	de	22	채택	
7	eg	25	버림	e, g 같은 덩어리
8	ef	27	채택	6개 = n−1 → **종료**
9	ab	29	보지 않음	

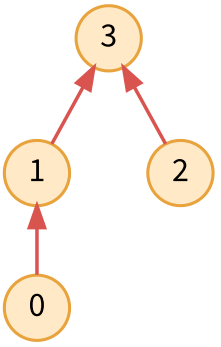
MST 합 = 10+12+15+16+22+27 = 102. "알파벳으로 나열" 문제는 채택된 간선만: `af, cd, bg, bc, de, ef`. 복잡도 **O(e log e)**(정렬이 대부분).

3. Union-Find (처음 질문했던 그 그림) 그래프II p.9~14

비유: 모임마다 대장(루트)이 한 명. `parent[i]` = "i가 보고하는 윗사람". **−1이면 i가 대장**. 처음엔 모두 −1.

find(x): **x에서 출발해 위로** 올라가 −1을 만나면 그 정점이 대장. 대장이 같으면 같은 모임 → Kruskal에서 **버림**.

union(a, b): 정점이 아니라 **대장끼리** 연결.



parent = [1, 3, 3, −1]
화살표 = "내 윗사람"

```
int set_find(int curr) {
    if (parent[curr] == -1) return curr;
    while (parent[curr] != -1) curr = parent[curr];
    return curr;
}
void set_union(int a, int b) {
    int root1 = set_find(a);
    int root2 = set_find(b);
    if (root1 != root2) parent[root1] = root2;   // a의 대장이 b의 대장 밑으로
}
/* kruskal 안 (p.18) */
uset = set_find(e.start);  vset = set_find(e.end);
if (uset != vset) { ...; set_union(uset, vset); }   // 다르면 채택, 같으면 버림
```

union 3단계 공식

① **첫 번째 정점의 대장**(uset)을 구한다 → ② **두 번째 정점의 대장**(vset)을 구한다 → ③ 다르면 `parent[uset] = vset`.
자기 검사: parent에 새로 적는 값은 항상 **parent가 −1인 정점**이어야 한다.

슬라이드 그림과 코드의 방향이 반대

방식	union(A, B) 결과 (A=0, B=1)	대장
교재 코드 `parent[root1] = root2`	parent[A] = 1, parent[B] = −1	B
슬라이드 그림 p.10~11 (B=0, H=2)	parent[A] = −1, parent[B] = 0	A

같은 모임 판단은 같지만 **배열 값은 다름**. 문제의 코드가 어느 방식인지 먼저 확인.

교재 코드로 p.20 그래프를 처리할 때 parent 변화 (실행으로 확인):

간선	find(첫째) / find(둘째)	결과	바뀐 칸	parent[0..6]
(시작)				[−1, −1, −1, −1, −1, −1, −1]
(5,0) 10	5 / 0	채택	parent[5]=0	[−1, −1, −1, −1, −1, **0**, −1]
(2,3) 12	2 / 3	채택	parent[2]=3	[−1, −1, **3**, −1, −1, 0, −1]
(6,1) 15	6 / 1	채택	parent[6]=1	[−1, −1, 3, −1, −1, 0, **1**]
(1,2) 16	1 / **3** (2→3)	채택	parent[1]=**3**	[−1, **3**, 3, −1, −1, 0, 1]

(6,3) 18	**3** (6→1→3) / 3	**버림**		그대로
(3,4) 22	3 / 4	채택	parent[3]=4	[−1, 3, 3, **4**, −1, 0, 1]
(6,4) 25	**4** (6→1→3→4) / 4	**버림**		그대로
(4,5) 27	4 / **0** (5→0)	채택	parent[4]=0	[−1, 3, 3, 4, **0**, 0, 1]

내가 틀렸던 것 (union-find는 3번 틀림, 마지막엔 맞힘)

① (1,2)에서 parent[1]=2라고 생각 → 2의 **대장 3**에 연결해야 함.

② find(0)을 "3에서 1을 거쳐 0" → find는 **물어본 정점에서 위로**: 0 → 1 → 3.

③ (4,0)에서 parent[4]=0 → 0의 대장 **3**: parent[4]=3.

④ (2,4)에서 parent[4]=3 → 바뀌는 칸은 **첫 번째 정점 2의 대장 3**: parent[3]=4.

⑤ [−1,−1,−1]에서 (0,2), (1,2) → **[2, 2, −1]** **정답!**

4. Prim 그래프II p.22~32

시작 정점에서 출발해 트리를 한 정점씩 키움. 매번 **트리에 인접한 정점 중 가장 싼 간선으로 연결된 정점**을 추가. `distance[v]` = v를 **지금의 트리**에 연결하는 가장 싼 간선.

```
for (i = 0; i < g->n; i++) {
    u = get_min_vertex(g->n);          // distance 최소인 미선택 정점
    selected[u] = TRUE;
    if (distance[u] == INF) return;
    for (v = 0; v < g->n; v++)
        if (g->weight[u][v] != INF)
            if (!selected[v] && g->weight[u][v] < distance[v])
                distance[v] = g->weight[u][v];    // ★ 간선 하나의 가중치만
}
```

추가된 정점	갱신 후 distance[0..6]	추가된 간선
0	[0, 29, ∞, ∞, ∞, 10, ∞]	(시작)
5	[0, 29, ∞, ∞, **27**, 10, ∞]	(0,5) 10
4	[0, 29, ∞, **22**, 27, 10, **25**]	(5,4) 27
3	[0, 29, **12**, 22, 27, 10, **18**]	(4,3) 22
2	[0, **16**, 12, 22, 27, 10, 18]	(3,2) 12
1	[0, 16, 12, 22, 27, 10, **15**]	(2,1) 16
6	(변화 없음)	(1,6) 15

추가 순서 **0 → 5 → 4 → 3 → 2 → 1 → 6**, 합 102 (p.31). **p.31 말풍선:** (1,6)이 15→19가 되면 (1,6) 대신 **(3,6) 18**이 들어가 합 **105**.

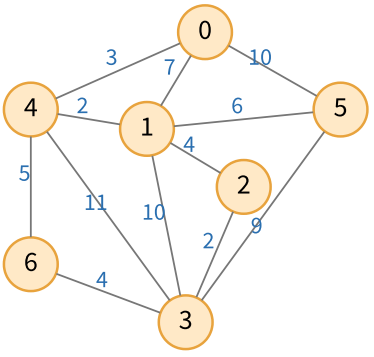
> **내가 틀렸던 것**
> 0, 5 추가 직후 distance[4]를 16이라고 씀 → **지금 트리 {0, 5}와 직접 연결된 간선만** 봄: 5-4 = **27**. 말풍선 문제는 합만 쓰고 "바뀐 간선"을 안 씀 → **묻는 것을 다 쓰기**.

복잡도: Prim $O(n^2)$, Kruskal O(e log e) → **희소 그래프는 Kruskal, 밀집 그래프는 Prim**이 유리(p.32).

5. Dijkstra 그래프II p.33~50

시작 정점 하나 → 모든 정점의 최단 거리. **음수 가중치 불가**(Bellman-Ford 필요, p.36). 집합 S(`found[]`) = 최단 거리가 확정된 정점.

왜 최소를 바로 확정해도 되나(p.37): S 밖의 다른 정점 w를 거쳐 u로 오면 이미 distance[w] ≥ distance[u]에서 출발하고, 가중치가 양수라 더 길어질 뿐. 음수가 있으면 이 논리가 깨짐.



그래프II p.34 (Dijkstra·Floyd 공통)

```
if (!found[w])
    if (distance[u] + g->weight[u][w] < distance[w])
        distance[w] = distance[u] + g->weight[u][w];   // ★ 누적 거리 (전역 distance에는 g-> 없음)
```

STEP	distance[0..6] (choose 전)	확정	그 정점 때문에 갱신
1	0 7 ∞ ∞ 3 10 ∞	**4** (3)	[1] 3+2=5, [3] 3+11=14, [6] 3+5=8
2	0 5 ∞ 14 3 10 8	**1** (5)	[2] 5+4=**9** (3: 15>14, 5: 11>10 그대로)
3	0 5 9 14 3 10 8	**6** (8)	[3] 8+4=12
4	0 5 9 12 3 10 8	**2** (9)	[3] 9+2=11
5	0 5 9 11 3 10 8	**5** (10)	없음
6	0 5 9 11 3 10 8	**3** (11)	(마지막)

확정 순서 **0→4→1→6→2→5→3**, distance[3]: ∞→14→12→11. **경로**(p.48~50): 갱신 시 `prev[w] = u`. prev = [−1, 4, 1, 2, 0, 0, 4] → 3의 경로 0→4→1→2→3(11), 6의 경로 0→4→6(8).

> **내가 틀렸던 것**
> "정점 1이 확정된 직후 갱신" 문제에 Floyd A^1 값(11, 17)을 씀. Dijkstra는 0→4→1 = 5를 이미 알고 있어 5+4=**9**. Floyd A^1은 경유지가 0,1로 제한돼서 7+4=11. **같은 칸이라도 알고리즘마다 중간값이 다름.**
> 빈칸에 `g->weight[u][w]` (Prim 답), 그다음엔 `g->distance[u]` (문법 오류) → 정답 `distance[u] + g->weight[u][w]`. **위 줄 if 조건에 답이 있음.**

6. Floyd 그래프II p.51~64

A^k[i][j] = 정점 **0~k만 경유지로** 쓸 때 i→j 최단 거리. A^{-1} = weight. k를 하나씩 늘리며 "k를 거치면 더 짧아지나?"만 확인.

```
for (k = 0; k < g->n; k++)          // ★ k가 반드시 가장 바깥
    for (i = 0; i < g->n; i++)
        for (j = 0; j < g->n; j++)
            if (A[i][k] + A[k][j] < A[i][j])
                A[i][j] = A[i][k] + A[k][j];   // (경로 버전: next[i][j] = k)
```

A^0(0 경유): 바뀌는 칸은 **A[4][5] = A[5][4] = 13**(3+10) 뿐 p.55

A^1(1 경유) p.57:

	0	1	2	3	4	5	6
0	0	7	11	17	3	10	∞
1	7	0	4	10	2	6	∞
2	11	4	0	2	6	10	∞
3	17	10	2	0	11	9	4
4	3	2	6	11	0	8	5
5	10	6	10	9	8	0	∞
6	∞	∞	∞	4	5	∞	0

A[4][5]: A^0에서 13 → A^1에서 **8**(2+6). 한 칸이 여러 단계에 걸쳐 계속 줄 수 있음.

단계	A^{-1}	A^0	A^1	A^2	A^3	A^4	최종
A[0][3]	∞	∞	17	13	13	**11**	**11**

최종 거리 = Dijkstra 최종값과 같아야 함 → 검산.

배열	저장하는 것	[0][3] 최종
A	최단 **거리**	11
next	경유하는 **정점 번호** (−1 = 바로 연결)	**4**

```
Path(0,3): next[0][3]=4 → Path(0,4) + 4 + Path(4,3)
           next[0][4]=-1 (직접)   Path(4,3): next[4][3]=2 → Path(4,2) + 2 + Path(2,3)
                                              Path(4,2): next[4][2]=1 → 4 → 1 → 2
결과: 0 → 4 → 1 → 2 → 3   (p.64 그림과 같음)
```

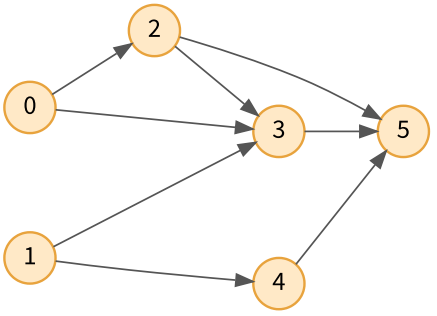
> **내가 틀렸던 것**
>
> next[0][3]을 17(거리)이라고 씀 → next는 **정점 번호**라 0~n−1 또는 −1만 가능. A[0][3] 최종을 12(Dijkstra STEP 4 중간값)라고 씀 → 최종은 **11**.

복잡도(p.61): Floyd $O(n^3)$. Dijkstra를 n번 돌려도 $O(n^3)$이지만 Floyd가 훨씬 간결. for문 순서를 i,j,k로 바꾸면 결과가 틀릴 수 있음(F).

7. 위상 정렬 그래프II p.65~72

방향 그래프에서 <u, v>가 있으면 u가 v보다 먼저 오도록 나열(선수 과목). 위상 순서는 **여러 개** 가능: (0,1,2,3,4,5), (1,0,2,3,4,5). (2,0,1,3,4,5)는 틀림(0→2인데 2가 먼저). **사이클이 있으면 불가능.**

알고리즘(p.67): 진입 차수 0인 정점 선택 → 출력 → 나가는 간선 삭제 → 반복. 정점이 남았는데 진입 차수 0인 정점이 없으면 사이클.



그래프II p.65 (0→2, 0→3, 1→3, 1→4, 2→3, 2→5, 3→5, 4→5)

인접 리스트는 **앞 삽입**이라 0: 3→2, 1: 4→3, 2: 5→3. 스택 사용(오른쪽 = top):

단계	pop(출력)	진입 차수 변화	in_degree[0..5]	스택
시작			[0, 0, 1, 3, 1, 3]	[0, 1]
1	**1**	4: 1→0 push, 3: 3→2	[0, 0, 1, 2, 0, 3]	[0, 4]
2	**4**	5: 3→2	[0, 0, 1, 2, 0, 2]	[0]
3	**0**	3: 2→1, 2: 1→0 push	[0, 0, 0, 1, 0, 2]	[2]
4	**2**	5: 2→1, 3: 1→0 push	[0, 0, 0, 0, 0, 1]	[3]
5	**3**	5: 1→0 push	[0, 0, 0, 0, 0, 0]	[5]
6	**5**			[ ]

출력 **1 4 0 2 3 5** (p.72, p.68 그림의 제거 순서와 같음).

다른 방식: 스택 대신 큐

같은 그래프에서 **0 1 2 4 3 5**(이것도 올바른 위상 순서). 문제 코드가 스택인지 큐인지 확인.

내가 틀렸던 것: 사이클이면 0을 반환하는가? → F라고 씀

`return (i == g->n);`에서 i = 출력한 정점 수. 사이클 0→1→2→0은 진입 차수가 모두 1이라 처음부터 스택이 비어 i=0 → 0 반환(실패). 정답 **T**. (1,4,0,2,5,3)이 틀린 이유는 "**간선 3→5가 있는데 5가 3보다 먼저 나와서**"라고 간선을 지목해 쓰기.

5장 · 총정리

헷갈리는 짝 · 복잡도 · T/F 모음 · 빈칸 단골 코드와 쪽수

1. 헷갈리는 짝

짝	A	B	구분하는 법
Prim ↔ Dijkstra	`distance[v] = g->weight[u][v]`	`distance[w] = distance[u] + g->weight[u][w]`	트리까지 **간선 하나** vs 시작점부터 **누적**
Floyd A ↔ next	거리	경유 정점 번호 (−1 = 직접)	next는 0~n−1, −1만 가능
Floyd 중간 ↔ Dijkstra 중간	A^k: 경유지 0~k로 제한	STEP: 확정된 정점 기준	최종값만 같음
union 방향	교재 코드: `parent[첫째 대장] = 둘째 대장`	슬라이드 그림: B의 부모가 A	문제 코드 확인
최대 ↔ 최소 히프	`>`, 큰 자식, `>=` 면 멈춤	`<`, 작은 자식, `<` 면 멈춤	부등호만 반대
BST 삭제 (자식 둘)	교재: 오른쪽 서브트리 최솟값	다른 방식: 왼쪽 서브트리 최댓값	둘 다 쓸 준비
허프만 비트	교재: 왼쪽 1, 오른쪽 0	보통: 왼쪽 0, 오른쪽 1	비트 반전
순회 자료구조	전위·중위·후위·DFS·위상정렬: **스택**	레벨 순회·BFS: **큐**	
경로 ↔ 단순 경로	이웃 사이 간선만 있으면 됨	+ 정점 반복 금지	
일반 DFS ↔ 연결 성분	`visited[v] = TRUE`	`visited[v] = count`	주변에 count++ 있는지
스레드 ↔ 진짜 자식	위로 가는 점선, is_thread=1 → 그대로 반환	아래로 가는 실선 → 오른쪽 서브트리 맨 왼쪽	

2. 시간 복잡도 한눈에

연산 / 알고리즘	복잡도	연산 / 알고리즘	복잡도
BST 탐색·삽입·삭제	O(h): 균형 O(log n), 경사 O(n)	DFS·BFS (인접 행렬)	$O(n^2)$
히프 삽입·삭제	O(log n)	DFS·BFS (인접 리스트)	O(n + e)
히프 정렬	O(n log n)	Kruskal	O(e log e)
정렬 안 된 배열 우선순위 큐	삽입 O(1), 삭제 O(n)	Prim	$O(n^2)$
정렬된 배열 우선순위 큐	삽입 O(n), 삭제 O(1)	Dijkstra / Floyd	$O(n^2)$ / $O(n^3)$

3. T/F 단골 문장 모음

문장	답	근거
공집합도 이진 트리이다	T	재귀적 정의 (트리 p.12)
포화 이진 트리는 완전 이진 트리이다 / 그 역	T / F	트리 p.17~18
스레드 이진 트리는 스택 없이 중위 순회 가능	T	후속자가 노드에 저장됨
BST 탐색은 최악에도 O(log n)	F	경사 트리 O(n)
히프에는 같은 키가 있을 수 없다	F	우선순위큐 p.9
최대 히프는 형제 사이에도 순서가 있다	F	느슨한 정렬
LPT는 최대 히프로 기계를 관리한다	F	가장 빨리 비는 기계 → 최소 히프
허프만 트리는 높이가 가장 낮은 트리이다	F	비용 최소 (93 vs 95)
무방향 그래프 차수 합 = 간선 수 × 2	T	그래프I p.11
인접 행렬 DFS는 O(n + e)	F	$O(n^2)$
Kruskal(탐욕적 방법)은 최적해를 보장한다	T	증명됨 (그래프II p.6)
Prim은 시작 정점에 따라 MST 합이 달라진다	F	MST 합은 유일
밀집 그래프에서는 Kruskal이 유리하다	F	밀집은 Prim
Dijkstra는 음수 가중치에서도 정확하다	F	Bellman-Ford (그래프II p.36)
Dijkstra 후 distance[v] = INF면 v로 가는 경로 없음	T	"v1→v7 경로 존재?" 문제
Floyd의 for문을 i, j, k 순으로 바꿔도 같다	F	k가 바깥이어야 함
사이클이 있으면 topo_sort는 0을 반환한다	T	`return (i == g->n)`

4. 빈칸 단골 코드와 교재 쪽수

함수	쪽수	자주 비우는 곳
TreeNode 구조체	트리 p.25	`struct TreeNode *left, *right;`
inorder / preorder / postorder	트리 p.40~41	printf 위치 (재귀 호출 앞·사이·뒤)
inorder_iter	트리 p.45	`for (; root; root = root->left) push(root);` , `root = root->right;`
level_order	트리 p.55	`enqueue(&q, ptr->left)`
evaluate / calc_dir_size	트리 p.61 / p.64	단말 조건, `root->data + left_size + right_size`
get_node_count / leaf / height	트리 p.65~66	`1 +` , `max`
find_successor / thread_inorder	트리 p.69 / p.70	`p->is_thread == TRUE` , `while (q->left != NULL) q = q->left;`

search (순환 / 반복)	트리 p.75 / p.76	`search(node->left, key)`
insert_node	트리 p.79	`if (node == NULL) return new_node(key);`
delete_node / min_value_node	트리 p.84~86	`root->left`, `min_value_node(root->right)`
insert_max_heap	우선순위큐 p.18, 25	`i / 2`, `i != 1`, `h->heap[i] = item`
delete_max_heap	우선순위큐 p.23, 26	`child < h->heap_size`, `child++`, `child *= 2`
heap_sort	우선순위큐 p.32	`a[i] = delete_max_heap(h)` (i는 n−1부터)
LPT main	우선순위큐 p.37~38	`m.avail + jobs[i] - 1`, `m.avail += jobs[i]`
insert/delete_min_heap, huffman_tree	우선순위큐 p.46~51	부등호 방향, `e1.key + e2.key`
insert_edge (인접 행렬 / 리스트)	그래프I p.22 / p.33	대칭 대입, `node->link = g->adj_list[u]`
dfs_mat / dfs_list	그래프I p.30 / p.33	`!visited[w]`, 재귀 호출
bfs_mat / bfs_list	그래프I p.40 / p.43	`visited[w] = TRUE; enqueue(&q, w);`
find_connected_component	그래프I p.45	`count++`, `visited[v] = count`
set_find / set_union	그래프II p.13~14	`curr = parent[curr]`, `parent[root1] = root2`
kruskal, compare	그래프II p.17~18	`x->weight - y->weight`, `uset != vset`
get_min_vertex / prim	그래프II p.28 / p.29	`distance[v] = g->weight[u][v]`
choose / shortest_path	그래프II p.43 / p.44	`distance[w] = distance[u] + g->weight[u][w]`
floyd	그래프II p.54	`A[i][j] = A[i][k] + A[k][j]`
topo_sort	그래프II p.69~70	`in_degree[node->vertex]++`, `push(&s, u)`, `return (i == g->n)`

빈칸 문법 3원칙

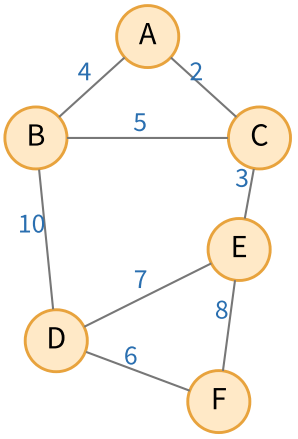
① `g->` 는 구조체 멤버(`weight`, `n`, `adj_mat`, `adj_list`)에만. 전역 배열(`distance`, `found`, `selected`, `visited`, `parent`, `A`)에는 안 붙음. ② 변수 이름은 **주변 코드 그대로** (Prim은 v, Dijkstra는 w). ③ **바로 위 if 조건**에 답이 있는 경우가 많음.

5. 최종 모의고사 (전체 범위 20문제)

⭐ 쉬움(1분) / ⭐⭐ 보통(2~3분) / ⭐⭐⭐ 어려움(5분). **권장 순서: A → D → B → C.** 정답은 다음 쪽.

공통 그래프 G (무방향, A=0 ~ F=5)
간선: A-B 4, A-C 2, B-C 5, B-D 10, C-E 3, E-D 7, D-F 6, E-F 8

방향 그래프 H: v1→v2, v1→v3, v2→v4, v3→v4, v4→v5, v6→v5 (v1=0 ~ v6=5)



그래프 G

A. T/F (⭐)

A1 포화 이진 트리는 완전 이진 트리이다.

A2 히프에는 같은 키 값을 가진 노드가 둘 이상 있을 수 없다.

A3 Prim 알고리즘은 시작 정점에 따라 MST의 가중치 합이 달라질 수 있다.

A4 인접 리스트로 표현된 그래프의 BFS 시간 복잡도는 O(n + e)이다.

A5 방향 그래프 H에서 v1 → v6 최단 경로가 존재한다.

D. 빈칸 (⭐, 교재를 찾아서)

D1 BFS: `visited[w] = TRUE; printf(…); ________(&q, w);`

D2 set_find: `while (parent[curr] != -1) curr = ________;`

D3 Floyd: `if (A[i][k] + A[k][j] < A[i][j]) A[i][j] = ________;`

D4 get_height: `height = 1 + ____(get_height(node->left), get_height(node->right));`

D5 BST 삭제(자식 둘): `TreeNode *temp = min_value_node(________);`

D6 위상 정렬: `if (in_degree[u] == 0) ____(&s, u);`

B. 객관식·단답 (⭐⭐)

B1 최대 히프 [20, 15, 18, 7, 10, 12]에서 삭제 1회 후 배열은? ① [18, 15, 12, 7, 10] ② [18, 12, 15, 7, 10] ③ [15, 18, 12, 7, 10] ④ [18, 15, 10, 7, 12]

B2 같은 원래 히프 [20, 15, 18, 7, 10, 12]에 25를 삽입한 뒤의 배열은?

B3 빈 BST에 50, 30, 70, 20, 40, 60, 80, 35, 45 순으로 삽입했을 때 전위 순회 결과는?

B4 B3 트리에서 30을 삭제하면 30 자리에 들어가는 값은? (교재 방식 / 왼쪽 서브트리 최댓값 방식 둘 다)

B5 H의 처음 in_degree 배열은? (v6, v1, v3, v2, v4, v5)는 올바른 위상 순서인가?

C. 중간 과정 (⭐⭐~⭐⭐⭐, 그래프 G)

C1 Kruskal이 채택하는 간선(순서대로, 알파벳), 버리는 간선, MST 합은?

C2 A에서 시작하는 Prim의 정점 추가 순서, E 추가 직후 distance[F], D 추가 직후 distance[F]는?

C3 A에서 시작하는 Dijkstra의 확정 순서와 distance[D]의 변화 과정은?

C4 C3에서 A → F의 최단 거리와 경로는?

C5 A에서 시작하는 인접 행렬 BFS(번호가 작은 이웃부터)에서 C를 꺼내 처리한 직후의 큐와 전체 방문 순서는?

C6 정점 3개, 간선 0-1 4, 1-2 1, 0-2 7에 Floyd 적용: A^{0}에서 바뀌는 칸, $A^{1}[0][2]$, next[0][2]는?

C7 ⭐⭐⭐ G에 교재 코드 Kruskal(`parent[uset] = vset`, 간선 검사 순서 (A,C), (C,E), (A,B), (B,C), (D,F), (E,D)) 적용 후 최종 parent[0..5]는?

6. 모의고사 정답과 풀이 (코드로 검증한 값)

번호	정답	풀이
A1	T	모든 레벨이 꽉 차 있으면 "왼쪽부터 빈칸 없이" 조건도 만족
A2	F	히프는 같은 키 허용 (BST와 다름)
A3	F	MST 가중치 합은 유일. 고르는 순서만 다름
A4	T	리스트는 실제 간선만 봄
A5	F	v6으로 들어가는 간선이 없음 (v6→v5만 있음, 화살표 방향 함정)
D1~D6	enqueue / parent[curr] / A[i][k] + A[k][j] / max / root->right / push	5장 빈칸 표의 쪽수에서 확인
B1	①	12를 루트로 → 자식 15, 18 중 큰 18과 교환 → 12는 3번, 자식 없음 → [18, 15, 12, 7, 10]
B2	[25, 15, 20, 7, 10, 12, 18]	7번 자리 → 부모 18(3번) 내림 → 부모 20(1번) 내림 → 루트
B3	50 30 20 40 35 45 70 60 80	VLR (중위는 오름차순 20 30 35 … 80으로 검산)
B4	교재 35 / 다른 방식 20	오른쪽 서브트리 {40, 35, 45}의 최솟값 35, 왼쪽 서브트리 {20}의 최댓값 20
B5	[0, 1, 1, 2, 2, 0], O	v1이 v2·v3보다, v2·v3이 v4보다, v4·v6이 v5보다 앞 → 모두 만족
C1	AC, CE, AB, DF, ED / 버림 BC / 22	정렬: AC2, CE3, AB4, BC5, DF6, ED7, EF8, BD10. BC는 {A,B,C,E} 안이라 사이클. 5개(n−1) 채택 후 종료 → EF, BD는 보지 않음. 2+3+4+6+7 = 22
C2	A C E B D F / 8 / 6	A 후 [B4, C2] → C 추가 후 E3 → E 추가 후 D7, F8 → B(4) → D(7) 추가 후 F = min(8, 6) = 6 → F
C3	C, B, E, D, F (A 다음) / ∞ → 14 → 12	C(2) 확정: E=5 / B(4): D=4+10=14 / E(5): D=5+7=12, F=13 / D(12): F 그대로 / F(13)
C4	13, A → C → E → F	prev[F] = E, prev[E] = C, prev[C] = A
C5	[D, E] / A B C D E F	[A] → [B, C] → B 처리 후 [C, D] → C 처리 후 [D, E] → [E, F] → [F] → []
C6	없음 / 5 / 1	A^0: 0 경유로 1→0→2 = 11 > 1이라 변화 없음. A^1: 0→1→2 = 4+1 = 5 < 7 → 갱신, next[0][2] = 1
C7	[2, 5, 4, 5, 1, −1]	(A,C): parent[0]=2 / (C,E): parent[2]=4 / (A,B): find(A)=0→2→4 = 4, parent[4]=1 / (B,C): find(C)=2→4→1 = 1 = find(B) → 버림 / (D,F): parent[3]=5 / (E,D): find(E)=4→1 = 1, find(D)=3→5 = 5 → parent[1]=5

채점 후 할 일

틀린 문제는 이 노트의 해당 장 표를 빈 종이에 다시 그려 보세요. 특히 C7(union 방향), C3(Dijkstra 중간값), B1·B2(히프 배열)는 이전에 틀렸던 유형입니다.